Supplementary Material

IRIS: A Real-World Benchmark for Inverse Recovery and Identification of Physical Dynamic Systems from Monocular Video

Anonymous ECCV submission

Paper ID 13771

A Dataset Statistics and Release Details

Table 1 provides a complete per-phenomenon, per-setting breakdown of the IRIS dataset. All ground-truth physical parameters are obtained through direct manual measurement using calibrated instruments (see Appendix C), with the exception of damping and friction coefficients, which are derived from trajectory fits as described in Appendix C. Each parameter entry records mean, standard deviation, minimum, and maximum values; where the standard deviation is zero, the measurement reflects a single precisely controlled experimental condition.

B Physical Model Derivations and ODE Bank

This appendix makes explicit which physical ODE governs each phenomenon, how the latent-space parameterisation (α, β) relates to physical constants, and where the pipeline's ODE form differs from the exact physical equation. The purpose is to prevent misreading of the latent parameters as direct physical measurements without the conversions listed below.

B.1 Phenomenon–ODE Correspondence Table

Table 2 summarises the mapping from IRIS/Delfys75 phenomena to physical ODEs and to the latent ODE form used in the pipeline. Where a mismatch exists, i.e. where the physical ODE is not a linear second-order oscillator but the pipeline treats it as one for the sake of a unified ODE bank, this is noted explicitly.

B.2 Design Choice: Unified Linear ODE Bank

The pipeline uses a unified second-order linear ODE $\ddot{z} + \beta\dot{z} + \alpha z = 0$ as the latent representation for all oscillatory and gravity-driven phenomena. This is a deliberate design choice for comparability across phenomena: the encoder learns

Table 1: **Full IRIS dataset statistics.** Each row is one setting; n is the number of videos. d_{cam} denotes camera-to-object distance. Measurement type: **D** = direct, **F** = fitted.

Phenomenon	Setting	n	Dur. (s)	Res.	GT params (measured)	Type
Single-body						
Dropping ball	drop_50	10	5	3840×2160	h_0=0.50 m, d_{cam}=1.94 m	D
	drop_100	10	5	3840×2160	h_0=1.00 m, d_{cam}=1.94 m	D
	drop_150	10	5	3840×2160	h_0=1.50 m, d_{cam}=1.94 m	D
Falling ball	big	10	8	3840×2160	g=9.81 m/s^2, r_0=0.11 m	D
	mid	10	8	3840×2160	g=9.81 m/s^2, r_0=0.07 m	D
	small	10	8	3840×2160	g=9.81 m/s^2, r_0=0.04 m	D
Sliding cone	cone_45	10	5	3840×2160	α=45°, hyp. =0.77 m, μ (fit)	D / μ:F
	cone_60	10	5	3840×2160	α=60°, hyp. =0.84 m, μ (fit)	D / μ:F
	cone_80	10	5	3840×2160	α=80°, hyp. =0.80 m, μ (fit)	D / μ:F
Pendulum	pend._20	10	150	3840×2160	θ_0=20°, L=0.50 m, ζ (fit)	D / ζ:F
	pend._45	10	150	3840×2160	θ_0=45°, L=0.50 m, ζ (fit)	D / ζ:F
	pend._90	10	150	3840×2160	θ_0=90°, L=0.50 m, ζ (fit)	D / ζ:F
Rotating cone	slow	10	8	3840×2160	Half-circle init., β (fit)	D / β:F
	mid	10	8	3840×2160	1-circle init., β (fit)	D / β:F
	fast	10	8	3840×2160	2-circle init., β (fit)	D / β:F
Multi-body						
Hitting cones	default	10	5	3840×2160	$d_{\text{ball-cones}}$=2.00 m, d_{cam}=2.20 m	D
Two mov. pendulums	pend._20	10	6	3840×2160	θ_0=20°, L_1=L_2=0.50 m	D
	pend._45	10	6	3840×2160	θ_0=45°, L_1=L_2=0.50 m	D
	pend._90	10	6	3840×2160	θ_0=90°, L_1=L_2=0.50 m	D
One stat. pendulum	pend._20	10	20	3840×2160	θ_0=20°, L_1=L_2=0.50 m	D
	pend._45	10	20	3840×2160	θ_0=45°, L_1=L_2=0.50 m	D
	pend._90	10	20	3840×2160	θ_0=90°, L_1=L_2=0.50 m	D
Total		**220**	–	–	–	–

to embed the observed motion into a one-dimensional latent z such that the linear ODE describes the latent trajectory, regardless of whether the underlying physical state (height, angle, radius) follows the same form exactly.

Dropping ball. The true physical ODE is $\ddot{h} = -g$ (constant acceleration with h in metres, t in seconds). The pipeline approximates this by $\ddot{z} + \alpha z = 0$ with $\alpha \approx g$; this approximation holds when the encoder maps the pixel position of the ball linearly to z and the observation window is short enough that the linear approximation of the trajectory remains valid. When $\beta \neq 0$, the term $\beta\dot{z}$ models aerodynamic drag. In practice, for the drop heights in IRIS (0.5–1.5 m), the drag term is small and $\alpha \approx g$ gives a physically meaningful estimate.

Falling ball (apparent radius). The ball falls away from the camera; the observable is its apparent radius $r(t) = r_0 f/(h_0 + \frac{1}{2}gt^2)$, where r_0 is the physical

Table 2: **Phenomenon–ODE correspondence.** Column "Physical ODE" is the exact governing equation. Column "Pipeline ODE form" is the form actually used by the encoder–physics block. Column "Latent → SI" lists the conversion from fitted (α, β) to physical parameters. $g = 9.81\,\mathrm{m/s^2}$ is treated as a known constant only for the pendulum length conversion; for dropping/falling ball it is the *estimated* quantity.

Phenomenon	Physical ODE	Pipeline ODE form	Latent → SI	Note
Dropping ball	$\ddot{z} = -g$ (constant accel.)	$\ddot{z} + \beta\dot{z} + \alpha z = 0$	$g = \alpha$	Unified linear form; β absorbs drag
Falling ball	$r(t) = r_0 f/(h_0 + \frac{1}{2}gt^2)$	$\ddot{z} + \beta\dot{z} + \alpha z = 0$	$g = \alpha$	Apparent radius; same latent form
Pendulum (small angle)	$\ddot{\theta} + \zeta\dot{\theta} + (g/L)\theta = 0$	$\ddot{z} + \beta\dot{z} + \alpha z = 0$	$L = g/\alpha,\ \zeta = -\beta$	Exact match for small angles
Rotating cone	$\ddot{\varphi} + \beta\dot{\varphi} + \alpha\varphi = 0$	$\ddot{z} + \beta\dot{z} + \alpha z = 0$	$\alpha_{\mathrm{SI}} = \alpha,\ \beta_{\mathrm{SI}} = \beta$	Exact match
Sliding cone	$\ddot{x} = g(\sin\alpha - \mu\cos\alpha)$	Constant-acceleration block	$\mu = \beta$; angle from metadata	Angle fixed from setting
LED decay	$\dot{z} = -\lambda z$	First-order decay block	$\gamma = \beta$	First-order; α unused
Torricelli drain	$\dot{z} = -k\sqrt{z}$	Torricelli block	$k = \alpha$	Nonlinear; separate block
Hitting cones	Momentum transfer (impact)	Graph ODE, coupling κ_{ij}	κ_{ij} (spring-like)	Simplified; see App. H
Two pendulums	$\ddot{\theta}_i + \zeta_i\dot{\theta}_i + (g/L_i)\theta_i + \kappa_{ij}(\theta_i - \theta_j) = 0$	Graph ODE, same form	$L_i = g/\alpha_i,\ \kappa_{ij}$ (coupling)	Active only during contact

ball radius, f the focal length, and h_0 the initial camera-to-ball distance. The latent z encodes this apparent size rather than height; the same second-order form is used, and the fitted α is compared to g after the same linear conversion. This is an approximation and is listed as such in Table 2.

Pendulum. For small oscillation angles $\theta_0 \leq 45°$, $\sin\theta \approx \theta$ and the linearised equation $\ddot{\theta} + \zeta\dot{\theta} + (g/L)\theta = 0$ is an exact match to the unified ODE form, with $\alpha = g/L$ and $\beta = \zeta$. For large angles ($\theta_0 = 90°$), the linearisation error is substantial ($\sin 90°/90° \approx 0.64$ in radians), and the fitted α will deviate from g/L accordingly. This is a **known modelling limitation**: on the pend._90 setting, higher parameter error is expected and should be interpreted as an identifiability challenge rather than a pipeline failure.

B.3 Dropping Ball vs. Falling Ball: Symbol Definitions

To avoid ambiguity between the two gravity-related phenomena:

- **Dropping ball**: camera is placed *to the side* of the drop. The observable is the *vertical pixel position* of the ball, which decreases linearly with $h(t) = h_0 - \frac{1}{2}gt^2$ (taking downward as positive). Ground truth: $h_0 \in \{0.50, 1.00, 1.50\}\,\mathrm{m}$, $d_{\mathrm{cam}} = 1.94\,\mathrm{m}$. Estimated parameter: g.
- **Falling ball**: camera is placed *directly above*, looking down. The ball falls *away* from the camera; the observable is the *apparent radius* $r(t) = r_0 f/(h_0 + \frac{1}{2}gt^2)$ (ball grows smaller). Ground truth: ball radii $r_0 \in \{0.04, 0.07, 0.11\}\,\mathrm{m}$; $g = 9.81\,\mathrm{m/s^2}$. Estimated parameter: g.

Symbol table: h_0 = initial drop height (m); d_{cam} = lateral camera distance (m); r_0 = physical ball radius (m); f = effective focal length (pixels or m); g = gravitational acceleration ($\mathrm{m/s^2}$); β = drag/damping coefficient; α = ODE stiffness coefficient.

C Ground-Truth Measurement Protocol

C.1 Directly Measured Parameters

Parameters labelled **D** in Table 1 are obtained through direct measurement before or after recording, independent of the video:

- **Heights and lengths** (h_0, L, L_1, L_2, hypotenuse length): measured with a rigid tape measure to the nearest millimetre. Each measurement is repeated three times; we report the mean. Uncertainty is ± 2 mm for lengths up to 1 m and ± 5 mm for longer spans.
- **Angles** (θ_0, α): set with an adjustable protractor or inclinometer calibrated to $\pm 0.5^\circ$. Cone incline angles are verified against the nominal setting after positioning.
- **Distances** (d_{cam}, $d_{\text{ball-cones}}$): measured with a laser distance meter (accuracy ± 2 mm).
- **Ball radius** (r_0): measured with digital calipers (resolution 0.01 mm); three measurements taken at different orientations, mean reported.
- **Gravitational acceleration** (g): taken as $9.81\,\text{m/s}^2$ (standard local value for the recording location; no significant variation expected across sessions).

C.2 Fitted Parameters: Damping and Friction

Parameters labelled **F** in Table 1 are *not* directly measurable in the same way as lengths or angles; they are instead estimated from recorded video trajectories:

Damping coefficients (ζ for pendulum, β for rotating cone). A reference video is selected per setting (the first clip of the session). The object's 2-D centroid or angular position is tracked frame by frame using a background-subtraction and blob-detection procedure. The resulting time series $\theta(t)$ is fitted to the analytical solution of the relevant damped oscillator ODE using nonlinear least squares (Levenberg–Marquardt). The fitted ζ (or β) is reported as ground truth for that setting. We use the same setting across all 10 trial videos. **Uncertainty:** we report the standard deviation of ζ across 10 independent fits; typical values are $\sigma_\zeta < 0.01\,\text{s}^{-1}$ for the pendulum and $\sigma_\beta < 0.005\,\text{s}^{-1}$ for the rotating cone.

Friction coefficient (μ for sliding cone). The video is segmented to track the cone's position along the slope. Acceleration a is estimated from a second-order polynomial fit to the position time series. The friction coefficient is then computed analytically as $\mu = \tan\alpha - a/(g\cos\alpha)$, where α is the measured incline angle. We average μ across 10 trial clips; standard deviation is reported. Typical $\sigma_\mu < 0.02$.

Limitations. Fitted parameters carry the error of the tracker and the fit model. For large oscillation angles ($\theta_0 = 90^\circ$), the small-angle approximation used in the linear-ODE fit is less accurate, and the reported ζ should be interpreted as the *effective* damping under the linearised model. We do not propagate tracking noise uncertainty into the final GT value; this is noted as a limitation and flagged in any per-setting identifiability discussion.

D Latent-to-Physical Parameter Calibration

The encoder maps raw pixel intensities to a latent z_t that is dimensionless. The physics block then fits α, β in this latent space. To obtain physical parameters (m, m/s^2, s^{-1}, etc.), the following per-phenomenon conversions are applied in the evaluation script (`compare_baseline_unified.py` and the IRIS counterpart):

Table 3: **Per-phenomenon calibration and physical parameter extraction.** α_{fit}, β_{fit} are the dimensionless fitted values. The rightmost column notes any normalisation or assumption.

Phenomenon	Extracted param.	Formula	Assumption / note
Dropping ball	g [m/s^2]	$g = \alpha_{\text{fit}}$	Latent = pixel height; dt in seconds
Falling ball	g [m/s^2]	$g = \alpha_{\text{fit}}$	Latent = apparent radius
Pendulum	L [m]	$L = g_{\text{true}}/\alpha_{\text{fit}}$	$g_{\text{true}} = 9.81\,\text{m/s}^2$ fixed
Pendulum	ζ [s^{-1}]	$\zeta = -\beta_{\text{fit}}$	Sign convention
Rotating cone	α, β	direct	Same ODE form
Sliding cone	μ	$\mu = \beta_{\text{fit}}$	Angle from metadata
LED decay	γ [s^{-1}]	$\gamma = \beta_{\text{fit}}$	First-order; α unused
Torricelli	k	$k = \alpha_{\text{fit}}$	Nonlinear Torricelli block

Temporal calibration. The pipeline receives pre-extracted video tensors with a fixed frame stride. The time step Δt passed to the integrator must match the physical frame interval; we set Δt to the inverse of the capture frame rate (60 fps $\rightarrow$ $\Delta t = 1/60$ s for IRIS; Delfys75 uses $\Delta t = 0.05$ s as in the original codebase). Mismatches between Δt and the true frame interval directly scale α and β by Δt^{-2} and Δt^{-1} respectively, which would bias all physical-parameter estimates. We verify that Δt is set consistently for each run.

Cross-phenomenon bias. The latent normalisation is phenomenon-specific: the encoder is trained from scratch on each clip, so the scale of z differs across phenomena. The calibration formulas above absorb this by expressing physical parameters directly in terms of α, β and known constants. However, no explicit pixel-to-metre calibration is performed; for phenomena where the latent encodes a pixel-level observable (e.g., apparent radius in the falling-ball case), small residual scale errors may remain if the encoder's internal normalisation drifts across clips. This is an acknowledged limitation; we recommend reporting relative rather than absolute parameter error for cross-phenomenon comparison.

E VLM Prompting Strategies and Ablations

The main paper reports three equation-identification strategies: VLM temporal reasoning (73.33% on Delfys75, 65.00% on IRIS), VLM describe-then-classify

(61.11% on Delfys75, 72.92% on IRIS), and a CNN video classifier (98.40% on Delfys75, 99.30% on IRIS). This appendix documents the exact prompt design and discusses additional strategies evaluated during development.

E.1 Frame Sampling

For all VLM calls, $m = 5$ representative frames are sampled from the clip at positions $\{0\%, 25\%, 50\%, 75\%, 100\%\}$ of the total duration. Earlier experiments used 3 frames (start, middle, end) and achieved ∼67% accuracy; adding the quartile frames increased accuracy to 73%. We use the same sampling for all strategies below.

E.2 Temporal Reasoning (Primary Variant)

The VLM (accessed via OpenRouter with a GPT-4V-class model) receives the 5 sampled frames as a sequence of images and the following system-level prompt:

> *"You are a physics expert. I will show you five frames from a video in temporal order. Your task is to identify the type of motion shown. Compare the first frame to the last frame to determine whether the motion is oscillatory, accelerating, decaying, rotating, or involves a collision. Then choose exactly one label from: {pendulum, free_fall, dropped_ball, sliding_block, led, torricelli}. Definitions: [one sentence per class]. Output only the label, nothing else."*

The key design choice is instructing the model to *compare early and late frames* to resolve temporal ambiguity (e.g., oscillatory pendulum vs. one-directional free fall, which can look identical in a single frame). This temporal-reasoning instruction is the main factor distinguishing this variant from the baseline VLM (which receives the same frames but without the comparison instruction), and accounts for most of the accuracy gain from ∼61% to 73% on Delfys75.

E.3 Describe-Then-Classify (Two-Call Variant)

This variant uses two sequential API calls. First, a description call asks the VLM to describe the motion in free text without any class labels provided. Second, a classification call sends that text description (not the images) to the model and asks it to assign one of the class labels. The intent is to use the VLM's language-reasoning strengths for the final label assignment, rather than relying on direct visual classification.

On Delfys75, this approach achieved 61.11% accuracy, lower than temporal reasoning (73.33%). Per-class breakdown: dropped_ball 100%, free_fall 0%, sliding_block 80%, torricelli 73%, led 60%, pendulum 53%. The collapse on free_fall (0%) is the dominant failure: the description step produces text that is ambiguous between free_fall and pendulum (e.g., "a ball moving downward"), and the classification step systematically maps this to pendulum.

Reversal on IRIS. Notably, the ranking of these two VLM strategies reverses on IRIS: describe-then-classify achieves 72.92% while temporal reasoning

drops to 65.00%. This reversal is attributable to the greater visual diversity of IRIS's eight phenomena, including three multi-body scenarios. The two-stage pipeline allows the VLM to first articulate complex dynamics (collisions, coupled oscillations, multiple moving objects) in free-form natural language, producing richer intermediate representations that facilitate disambiguation between single-body and multi-body phenomena. By contrast, the single-call temporal reasoning prompt, which was designed around the six Delfys75 classes, struggles to capture the novel motion patterns in IRIS within a single classification step. This finding suggests that no single VLM prompting strategy generalizes reliably across benchmarks, and motivates research on adaptive or ensemble-based equation routing.

E.4 Additional Strategies Evaluated

The baseline pipeline from Garcia et al. relies exclusively on path-based equation selection, where the correct ODE family is provided via folder structure at inference time. This is an oracle condition that cannot generalise to new recordings without manual labelling. A core motivation for introducing IRIS is to benchmark automatic equation-routing strategies that remove this requirement. VLM-based routing is a natural candidate because it operates zero-shot without retraining, can handle novel phenomena, and leverages broad visual and physical knowledge acquired during pretraining. The strategies below were explored during development to understand the trade-offs across different automation approaches.

Chain-of-thought (CoT) prompting. We evaluated a variant that explicitly asks the model to reason step by step before outputting the label ("First describe what you observe, then reason about the physics, then state the label"). Accuracy was marginally higher than the baseline VLM (∼65%) but lower than temporal reasoning, with significantly higher token cost per call.

Few-shot prompting. Providing one example image per class in the prompt context improved robustness for visually similar classes (dropped_ball vs. free_fall) but was impractical at scale due to context length limits and per-call cost.

Self-consistency. Sampling the model $k = 5$ times per clip and taking the majority vote improved stability on ambiguous clips but increased cost proportionally, without a meaningful accuracy gain over temporal reasoning.

VLM fine-tuning. We attempted LoRA fine-tuning and a frozen-VLM plus classifier-head approach, both of which collapsed to predicting a single class for every video (∼16.7% accuracy, equal to random for 6 classes). This is likely due to the small dataset size (90 training videos) relative to the VLM's parameter count. Fine-tuning is therefore not recommended and is not used in any reported result.

Video classifier (ResNet-18 + temporal pooling). A lightweight ResNet-18 backbone with mean temporal pooling is trained separately on each benchmark: 6-class on Delfys75 and 8-class on IRIS, using their respective training splits. On Delfys75, the classifier achieves 98.40% accuracy; on IRIS, it achieves 99.30% accuracy. These results confirm that supervised classifiers attain near-perfect in-distribution performance on both benchmarks, but this comes at the cost of

requiring retraining whenever the phenomenon set changes. Unlike the zero-shot VLM strategies, the CNN cannot handle novel phenomena without collecting labelled data for the new classes. Disentangling the effect of distribution shift from scenario complexity would require evaluating the Delfys75-trained CNN on IRIS's overlapping phenomena (dropped ball, pendulum, sliding cone); this cross-distribution evaluation is left for future work.

E.5 VLM Confusion Matrix

Table 4 reproduces the confusion matrix for the temporal-reasoning VLM on Delfys75, included here for convenience when reading the prompting discussion above. The primary failure mode is free_fall being predicted as pendulum (10 out of 15 videos): a ball shrinking in apparent size as it falls away from the camera is visually indistinguishable from an object receding in oscillatory motion when only 5 frames are shown without metric scale context. dropped_ball is confused with free_fall (5 of 15) for a symmetric reason: both show a ball at increasing pixel displacement, and the direction (lateral vs. depth-wise) is only apparent from late frames. LED, sliding block, and Torricelli classify without error due to unambiguous visual signatures (brightness change, translational inclined motion, water level decrease).

Table 4: **Confusion matrix, temporal-reasoning VLM on Delfys75.** Rows = ground truth, columns = predicted. Diagonal = correct.

	drop.	**free**	**led**	**pend.**	**slide**	**torr.**	Σ
dropped_ball	**10**	5	0	0	0	0	15
free_fall	0	**0**	0	10	5	0	15
led	0	0	**15**	0	0	0	15
pendulum	2	2	0	**11**	0	0	15
sliding_block	0	0	0	0	**15**	0	15
torricelli	0	0	0	0	0	**15**	15
Acc	66/90 = **73.3%**						

F Implementation Details

F.1 Encoder Architecture

Following the original pipeline [1], the encoder ϕ is a lightweight MLP operating on per-frame pixel intensities. Input frames are resized to 56×100 (grayscale) and flattened to a 5600-dimensional vector. The MLP has two hidden layers of 256 units each with ReLU activations, and an output of dimension $d = 2$ (mean μ_z and log-variance $\log \sigma_z^2$ for the VAE reparameterisation). The latent state $\hat{z}_t$ is sampled as $\hat{z}_t = \mu_z + \sigma_z \cdot \epsilon$, $\epsilon \sim \mathcal{N}(0, 1)$.

For multi-object IRIS phenomena (hitting cones, two pendulums), the encoder outputs $2N$ values, reshaped to $N \times 2$ per frame. All N objects share encoder weights.

F.2 Physics Block

Each ODE family has a dedicated physics block with learnable scalars $\boldsymbol{\gamma} = (\gamma_0, \gamma_1)$ initialised to $(0.5, 0.05)$. For the second-order ODE $\ddot{z} + \gamma_1 \dot{z} + \gamma_0 z = 0$, the discrete Euler-Cromer step is:

$$\dot{z}_{t+1} = \dot{z}_t + \Delta t \cdot \ddot{z}_t, \tag{1}$$

$$z_{t+1} = z_t + \Delta t \cdot \dot{z}_t + \Delta t^2 \cdot \ddot{z}_t, \tag{2}$$

where $\ddot{z}_t = -\gamma_1 \dot{z}_t - \gamma_0 z_t$. The critical fix relative to the original codebase is the inclusion of the $\Delta t^2 \cdot \ddot{z}_t$ term in the position update: without it, $\partial z_{t+1}/\partial \gamma = 0$ and physics parameters receive no gradient signal during training.

F.3 Training

Each clip is trained independently with Adam; no parameters are shared across clips. Hyperparameters: learning rate 10^{-3} for the encoder, 10^{-2} for $\boldsymbol{\gamma}$; batch size = all temporal windows of the clip; 500 epochs. The one-step loss is:

$$\mathcal{L}_{\text{1-step}} = \frac{1}{M} \sum_{i=1}^{M} \|\hat{\mathbf{z}}_i - \tilde{\mathbf{z}}_i\|^2 + \lambda_{\text{KL}} \, \mathcal{L}_{\text{KL}},$$

and the multi-step loss replaces the first term with $\sum_{k=1}^{K} w_k \|\hat{\mathbf{z}}_{t+k} - \tilde{\mathbf{z}}_{t+k}\|^2$ (main paper, Eq. 2), with $K = 5$ and weights $w = [1.0, 1.0, 0.5, 0.5, 0.25]$. In both cases $\lambda_{\text{KL}} = 0.01$.

All runs use a fixed random seed (seed = 42) for reproducibility. Run outputs are written to CSV with columns `run, alpha, beta, max_z, min_z, z0, z1` (one row per clip) and are processed by `compare_baseline_unified.py` for parameter extraction and GT comparison.

G Full IRIS Parameter Estimation Results

Table 5 reports parameter MAE on IRIS comparing two training objectives applied to the **+Corrected** configuration (gradient-corrected Euler, path-based equation selection): (a) one-step loss (*1-step*) and (b) multi-step rollout loss with K=5 (*Multi*). This corresponds to ablation configurations 2 and 3 in the main paper (Sec. 6.1); it isolates the effect of the loss function independently of VLM routing. $\Delta = \text{MAE}_{\text{multi}} - \text{MAE}_{\text{1-step}}$ (negative = multi-step wins).

Table 5: **Parameter MAE on IRIS: 1-step vs. multi-step loss** (both with corrected Euler, path-based selection; ablation configs 2 vs. 3 of the main paper). Δ=MAE$_{\text{multi}}$−MAE$_{\text{1-step}}$; negative = multi-step wins. Green = lower MAE; red = catastrophic ($|\Delta|$>10).

Dynamics	Setting	Param	GT	1-step	Multi.	Δ
Single-body						
Dropping ball	drop_50	g [m/s^2]	9.81	**1.04**	6.88	+5.84
	drop_100	g [m/s^2]	9.81	**2.26**	8.28	+6.02
	drop_150	g [m/s^2]	9.81	**3.83**	8.86	+5.04
Falling ball	big	g [m/s^2]	9.81	**4.26**	6.40	+2.14
	mid	g [m/s^2]	9.81	**6.63**	8.63	+2.01
	small	g [m/s^2]	9.81	**4.13**	6.88	+2.74
Pendulum	pend._20	L [m]	0.50	0.56	**0.40**	−0.16
	pend._45	L [m]	0.50	**0.34**	1.16	+0.82
	pend._90	L [m]	0.50	**0.60**	21.95	+21.34
Rotation	fast	α	0.10	1.54	**2.46**	−0.64
		β	0.08	**0.88**	1.28	+0.39
	mid	α	0.10	3.03	**3.98**	−1.21
		β	0.05	**0.39**	1.38	+0.99
	slow	α	0.10	1.59	**4.63**	−2.11
		β	0.03	0.45	**0.06**	−0.39
Sliding cone	cone_45	α [deg]	45.0	**0.00**	**0.00**	0.00
	cone_60	α [deg]	60.0	**0.00**	**0.00**	0.00
	cone_80	α [deg]	80.0	**0.00**	**0.00**	0.00
Multi-body						
Two mov. pendulums	pend._20	L_1, L_2 [m]	0.50	**1.14**	741.1	+740.0
	pend._45	L_1, L_2 [m]	0.50	**1.63**	164.1	+162.5
	pend._90	L_1, L_2 [m]	0.50	**3.01**	149.9	+146.9
One stat. pendulum	pend._20	L_1, L_2 [m]	0.50	**1.38**	65.4	+64.0
	pend._45	L_1, L_2 [m]	0.50	**0.77**	96.8	+96.0
	pend._90	L_1, L_2 [m]	0.50	**1.06**	164.8	+163.8

Key observations.

(1) *Sliding cone angle:* both variants recover the correct angle exactly because it is taken directly from the setting metadata, not from training.

(2) *Gravity (dropping/falling ball):* the 1-step variant outperforms multi-step consistently here, mirroring the Delfys75 finding (main paper, Table 5). The multi-step loss accumulates integration error over K=5 horizons, inflating the α estimate when the encoder is not yet close to the physical trajectory.

(3) *Pendulum pend._90:* the catastrophic multi-step failure ($\Delta = +21.34$) reflects large-angle nonlinearity. The linearised ODE $\ddot{z} + \beta\dot{z} + \alpha z = 0$ is a poor model

for $\theta_0 = 90°$ (Appendix B), and multi-step rollout amplifies this approximation error over K steps.
(4) *Multi-body pendulums:* both variants yield large absolute errors. This is expected given the simplified contact model (Appendix H) and the brevity of the contact window relative to the total clip. The multi-step loss further destabilises training for these under-constrained scenarios. These rows serve as a baseline target for future multi-body methods.
(5) *Hitting cones (omitted):* the hitting-cones scenario is excluded from this table because the effective coupling coefficient κ has no independently measured ground-truth value; it is an artefact of the spring-like simplification (Appendix H) rather than a physical observable. Qualitatively, both configurations produce near-zero κ estimates on this scenario, consistent with the finding that the contact window is too brief relative to the clip duration for the coupling term to receive a meaningful gradient signal.

H Multi-Body Contact Modeling: Details and Limitations

The graph-structured physics block models inter-object forces via a learnable spring-like coupling term $\kappa_{ij}(z_i - z_j)$ (main paper, Eq. 1). This is a deliberate simplification motivated by two goals: (a) the architecture reduces to the single-body ODE exactly when no edges are present ($N = 1$), enabling a unified codebase; (b) the coupling term is differentiable and allows gradient-based identification of interaction strength.

Known limitations of this model. Real physical impacts involve:

- *Restitution*: velocity reversal at contact, characterised by the coefficient of restitution $e \in [0, 1]$, which is not representable by a static coupling coefficient.
- *Finite contact duration*: impacts in rigid-body mechanics are instantaneous (or very brief), while the spring model spreads the force over a continuous time window proportional to $1/\kappa_{ij}$.
- *Friction at contact*: the tangential impulse during collision is not modelled.

As a consequence, the fitted κ_{ij} for hitting-cone and two-pendulum scenarios is an *effective* coupling coefficient that conflates all of the above effects into a single linear term. Parameter estimates for multi-body IRIS scenarios should therefore be interpreted as "effective coupling strength" rather than as physically interpretable constants, and are primarily useful as a benchmark target for future methods.

Identifiability note. The coupling parameter κ_{ij} is only observable during the brief contact interval. For the two-pendulum and one-static-pendulum settings, this interval represents a small fraction of the total clip (≤ 1 s out of 6–20 s); the latent loss over the full clip is dominated by the non-contact phase, where κ_{ij} is invisible. This explains the large MAE values in Table 5 for multi-body settings. Future work should consider contact-segmented training windows or event-conditioned physics blocks.

I Identifiability Analysis

Motivation. Parameter identifiability, whether a unique parameter vector $\boldsymbol{\gamma}^*$ can be recovered from the observed data, is a fundamental concern for any parameter estimation method. We provide a preliminary analysis using two proxies: (1) gradient norms during training (do physics parameters receive gradient signal?), and (2) estimate variance across repeated runs.

I.1 Gradient Norms

The critical Euler fix (Appendix F.2) ensures that $\partial z_{t+1}/\partial \boldsymbol{\gamma} \neq 0$. We confirmed this empirically: without the fix, γ_0 and γ_1 remained at their initial values $(0.5, 0.05)$ for all 500 epochs; gradient norms were $< 10^{-8}$ throughout. After the fix, on a representative dropped_ball clip, gradient norms are $\|\nabla_\gamma \mathcal{L}\|_2 \approx 0.3$–$1.5$ at epoch 1, decaying to ≈ 0.05–0.2 at convergence, confirming that training is meaningful for the physics block.

Parameter trajectories. On a dropped_ball clip, γ_0 evolves from $0.5 \to 4.65$ over 500 epochs (target: $g \approx 9.81$, relative error $\sim$50%); γ_1 (drag) evolves from $0.05 \to 1.11$. For pendulum clips, γ_0 converges to values implying $L = g/\gamma_0 \in [0.5, 1.6]$ m, bracketing the true length of 0.5 m for pend._20 and pend._45 but overestimating for pend._90 (consistent with the large-angle linearisation error).

I.2 Estimate Variance Across Trials

Table 6 reports the mean and standard deviation of estimated parameters across 10 repeated trials per setting (IRIS baseline, one-step loss).

Table 6: **Estimate variance across 10 trials per IRIS setting.** $\bar{\gamma}$ = mean estimate; σ = std; GT = ground truth. Low σ indicates stable convergence; high σ indicates identifiability difficulties.

Dynamics	Setting	Param	GT	$\bar{\gamma}$	σ
Dropping ball	drop_50	g	9.81	8.77	0.42
Dropping ball	drop_100	g	9.81	7.55	1.21
Dropping ball	drop_150	g	9.81	5.98	2.04
Pendulum	pend._20	L [m]	0.50	1.06	0.88
Pendulum	pend._45	L [m]	0.50	0.84	0.41
Pendulum	pend._90	L [m]	0.50	1.10	0.73
Sliding cone	cone_45	α [deg]	45.0	45.0	0.00
Rotation	slow	β	0.03	0.48	0.09

Observations. (1) Sliding cone angle is recovered perfectly across all trials (metadata-based, variance = 0). (2) Gravity estimates are stable for short drops

(drop_50: $\sigma = 0.42$) but increasingly variable for longer drops, where the quadratic trajectory requires more clip length to constrain α. (3) Pendulum length shows moderate variance, consistent with the identifiability challenge of disentangling $\gamma_0 = g/L$ from the encoder's internal scale. Full identifiability analysis, including Fisher information matrix computation and Hessian condition numbers for $\boldsymbol{\gamma}$, is left for future work.

References

[1] Alejandro Castañeda Garcia, Jan Warchocki, Jan van Gemert, Daan Brinks, and Nergis Tomen. "Learning physics from video: Unsupervised physical parameter estimation for continuous dynamical systems". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 27924–27933.